Пространственные структуры данных

3D зрение

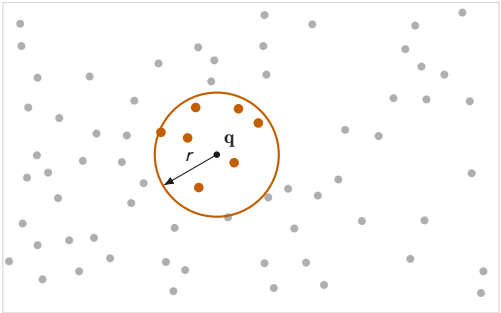
Лаборатория компьютерной графики и мультимедиа
ВМК МГУ && ФИИ МГУ

12 октября 2026 г.

Задача: все точки внутри шара

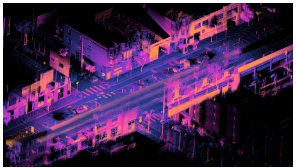
Дано: множество точек
$P = \{\mathbf{p}_1, \ldots, \mathbf{p}_N\} \subset \mathbb{R}^3$.
Запрос $(\mathbf{q}, r)$: найти

$$S(\mathbf{q}, r) = \{\, i \;:\; \|\mathbf{p}_i - \mathbf{q}\| \le r \,\}.$$

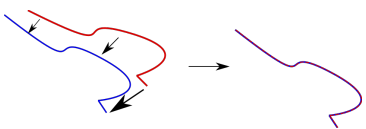


- Запросов много, часто по одному на точку: $M \approx N$.
- Ответ мал: $K = |S| \ll N$, обычно десятки точек.
- Точки обновляются часто (новый скан) или редко (готовое облако). От этого зависит, что важнее: быстрое построение или быстрый запрос.

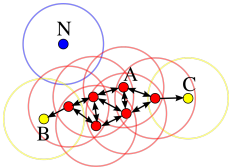
Где это нужно



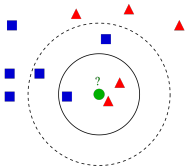
Обработка облаков точек лидара



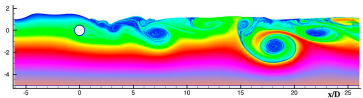
Совмещение облаков точек (ICP)



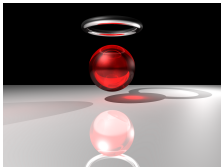
Кластеризация (DBSCAN)



Классификация по соседям (kNN)



Частицы и жидкости (SPH)



Фотонные карты

Изображения: Wikimedia Commons — D. L. Lu (CC BY 4.0); Biggerj1, Asoutoiglesias, P. Karandikar (CC BY-SA 4.0); Chire, AnAj (CC BY-SA 3.0).

Наивное решение: перебор

```
for (int i = 0; i < N; i++)
  if (length2(p[i] - q) <= r*r) // без sqrt
    result.push_back(i);
```

- Один запрос: $O(N)$.
- Все запросы: $O(NM) = O(N^2)$.

Пример: облако из $N = 10^6$ точек, у каждой $K \approx 50$ соседей.

- перебор: 10^{12} проверок;
- полезных из них: $5 \cdot 10^7$, то есть $0{,}005\%$.
- Память: 0 байт сверх самих точек.

Добавление ускоряющей структуры:

Скорость

- запрос за $O(log(N))$ вместо $O(N)$;
- построение за $O(kN)$ или $O(N \log N)$.

Память

- сколько-то сверх самих точек;
- может зависеть от размера области.



Какие способы решения этой задачи вы знаете?

План лекции

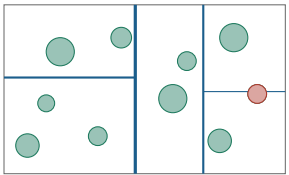
1. **Регулярная сетка** — простейшая структура, запрос за $O(1)$ ячеек.
2. **Пространственное хэширование** — сетка без пустых ячеек и без ограничения на размер области.
3. **Z-индекс** — порядок ячеек в памяти и переход к октодеревьям.
4. **kd-деревья и BSP** — разбиение плоскостью в произвольном месте.
5. **BVH** — разбиение объектов вместо пространства.
6. **Библиотеки** — OpenVDB, Embree.
7. **Высокие размерности** — почему всё это ломается при $d \sim 100$.
8. **Векторный поиск** — как работают векторные базы данных.

Каждая следующая структура исправляет недостаток предыдущей.

Основная идея ускоряющих структур

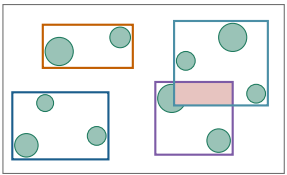
Дешёвым тестом отбросить сразу группу точек, которые заведомо не попадут в шар, и точно проверять только оставшихся **кандидатов**.
Группы можно строить двумя принципиально разными способами.

Разбиение пространства
области не пересекаются, объект может попасть в несколько областей



kd-дерево: красный объект — в обеих половинах

Разбиение объектов
каждый объект ровно в одной группе, объёмы групп могут пересекаться



BVH: красная зона — пересечение боксов

1

РАЗДЕЛ

Регулярная сетка

Регулярная сетка

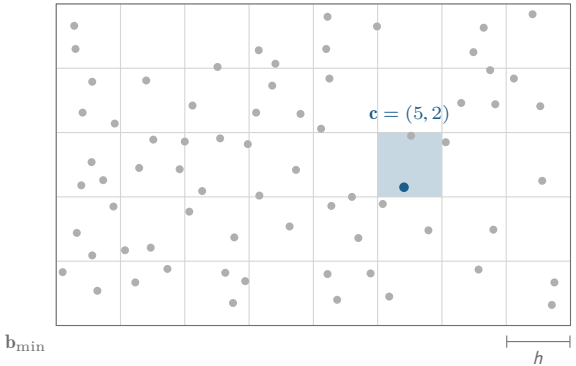
Ограничивающий бокс $[\mathbf{b}_{\min}, \mathbf{b}_{\max}]$ делим на кубические **ячейки** со стороной h:

$$n_x = \left\lceil \frac{b_{\max,x} - b_{\min,x}}{h} \right\rceil, \ \ldots$$

Ячейка точки — без сравнений, за $O(1)$:

$$\mathbf{c} = \left\lfloor \frac{\mathbf{p} - \mathbf{b}_{\min}}{h} \right\rfloor,$$

$$\mathrm{id} = c_x + n_x\,(c_y + n_y\,c_z).$$



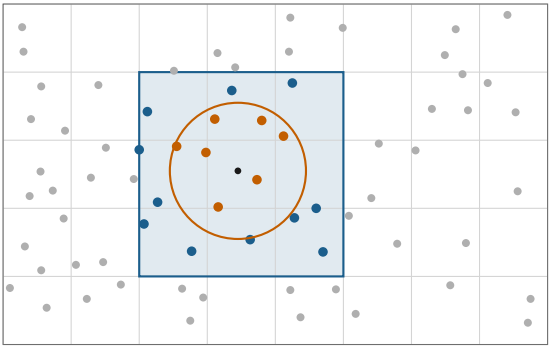
Запрос: какие ячейки проверять

Шар целиком лежит в боксе $[\mathbf{q} - r,\ \mathbf{q} + r]$, поэтому достаточно ячеек с индексами от

$$\mathbf{c}_{\text{lo}} = \text{cell}(\mathbf{q} - r) \ \text{ до } \ \mathbf{c}_{\text{hi}} = \text{cell}(\mathbf{q} + r).$$

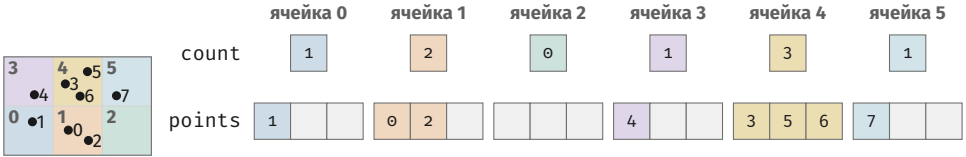
При $h = r$ это блок 3×3 на плоскости и $3 \times 3 \times 3 = 27$ ячеек в 3D.

- — результат запроса
- — проверены зря
- — не просмотрены вовсе



Хранение в одном буфере: фиксированная ёмкость

Каждой ячейке заранее отводим M мест в общем буфере и храним счётчик заполненных.



k-я точка ячейки c лежит в `buffer[c·M + k]`, где $k <$ `count[c]`. В примере $M = 3$.

- \+ Построение в один проход.
- \+ Точку можно добавить без перестройки.
- – Память $C \cdot (M + 1)$ чисел, многие места пустуют.
- – Больше M точек в ячейке — переполнение.

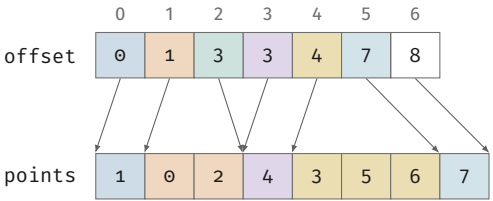


**Как сократить расход памяти,
оставаясь в рамках регулярной сетки?**

Хранение в одном буфере: смещения

Каждой ячейке — только смещение `offset[c]`: оно указывает на голову её списка в отдельном буфере, а конец списка — это следующее смещение `offset[c+1]`.



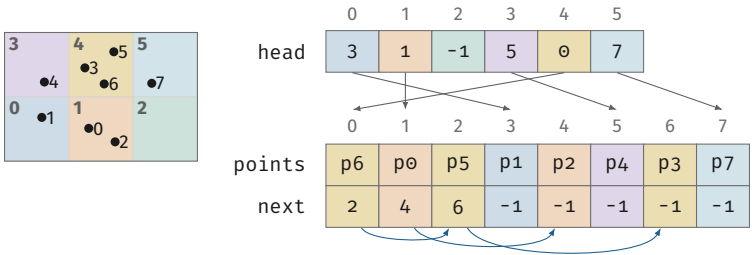


Точки ячейки c: `buffer[offset[c]]` … `buffer[offset[c+1]-1]`. Пустая ячейка 2: `offset[2] = offset[3]`.

+ Память: N индексов;
+ Прострая структура данных.
– Вставка сортировкой: $N log(N)$

Хранение в одном буфере: связные списки

Каждой ячейке — голова списка `head[c]`. В буфере `points` лежат пары (точка, `next`): точки разных ячеек перемешаны, а `next` ведёт к следующей точке той же ячейки.



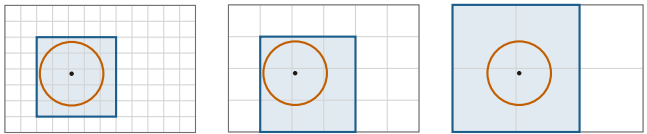
Обход ячейки c: `for (k = head[c]; k != -1; k = next[k])`. Ячейка 4: слоты $0 \to 2 \to 6$.

- \+ Память $N + C$ чисел.
- \+ Вставка за $O(1)$.
- – Точки ячейки разбросаны: больше кэш-промахов.



Как амортизировать кэш-промахи?

Выбор размера ячейки

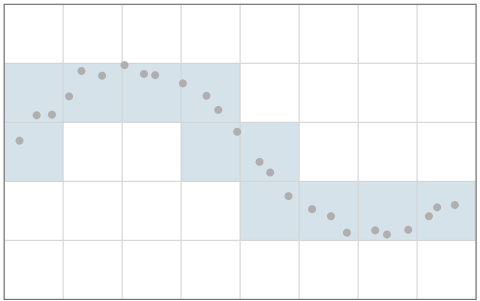


$h = r/2$ $h = r$ $h = 2r$

3D	$h = r/2$	$h = r$	$h = 2r$
ячеек в запросе	$5^3 = 125$	$3^3 = 27$	$2^3 = 8$
объём кандидатов	$15{,}6\,r^3$	$27\,r^3$	$64\,r^3$
доля полезных кандидатов	27%	16%	6,5%
число ячеек в сетке	$\times 8$	$\times 1$	$\times 1/8$

Мельче ячейка — меньше лишних точек, но больше ячеек и памяти. На практике $h = r$ или $h = 2r$.

Проблема 1: память на пустые ячейки



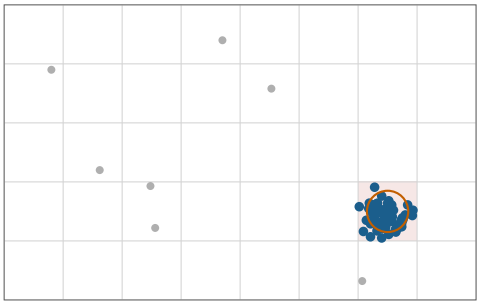
Скан улицы: заняты 11 из 40 ячеек, память — на все 40.

- Число ячеек $C = n_x n_y n_z$ зависит от размера области, а не от числа точек.
- Скан улицы 100 м, радиус поиска 1 см: 10^{12} ячеек, только массив `offset` — 4 ТБ.
- Растущая область (данные приходят потоком) в сетку не помещается.

Решение

Хранить только непустые ячейки: хэш-таблица размера $O(N)$.
$\Rightarrow$ **раздел 2, пространственное хэширование.**

Проблема 2: неравномерное распределение



«Чайник на стадионе»: 60 из 67 точек в одной ячейке.

- Запрос малого радиуса внутри кластера проверяет весь кластер: снова $O(N)$.
- Уменьшить h ради кластера — получить миллионы пустых ячеек на остальном поле.
- Сетка настроена на один масштаб: и плотности, и радиуса запроса.

Решение

Разбивать пространство адаптивно: мельче там, где точек много.
$\Rightarrow$ **иерархические структуры, разделы 4–6.**



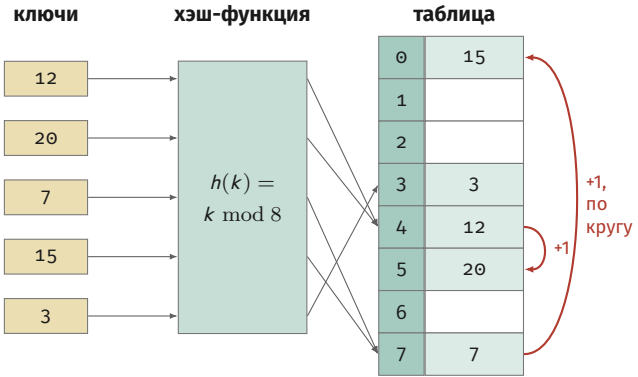
Как решать эти проблемы?

2

РАЗДЕЛ

Пространственное хэширование

Хэш-таблица с линейным зондированием



- Таблица — массив из T слотов.
- Хэш-функция превращает ключ в номер слота.
- Два ключа в одном слоте — коллизия: кладём в следующий свободный (20 → 5, 15 → 0).
- Поиск идёт так же до пустого слота. Ищем 28: слот 4, 5, 6 — пусто, ключа нет.

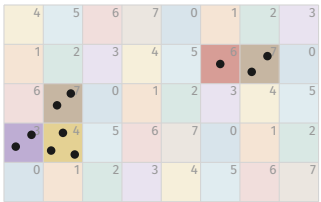
Пока таблица заполнена не больше чем наполовину ($T \geq 2n$), поиск в среднем смотрит 1–3 слота: $O(1)$.



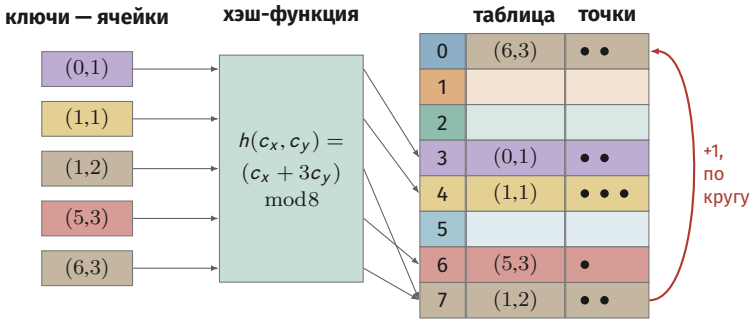
Какие ещё варианты зондирования/пробинга Вы знаете?

Пространственная хэш-таблица

Та же хэш-таблица: ключ — ячейка, значение — её точки. Память $O(N)$.



у любой ячейки есть слот: цвет и число



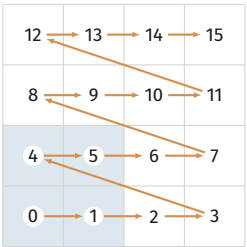
На практике в 3D: $h = (c_x \cdot 73856093) \oplus (c_y \cdot 19349663) \oplus (c_z \cdot 83492791) \bmod T$ [Teschner 2003].

3

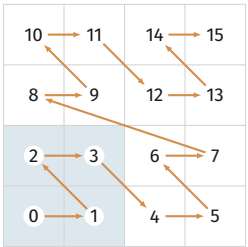
РАЗДЕЛ

Z-индекс (коды Мортона)

Порядок обхода ячеек



по строкам



Z-порядок

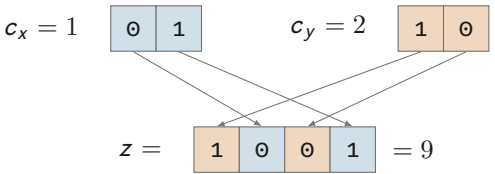
Синий блок 2×2:

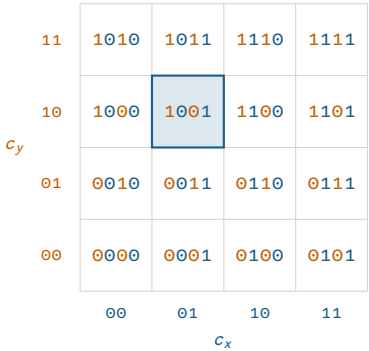
- по строкам — номера 0, 1, 4, 5: разбросаны;
- Z-порядок — номера 0, 1, 2, 3: подряд.

В Z-порядке подряд идёт любой выровненный блок 2×2, 4×4, ... Близкие в пространстве ячейки обычно близки и по номеру.

Как вычисляется Z-индекс

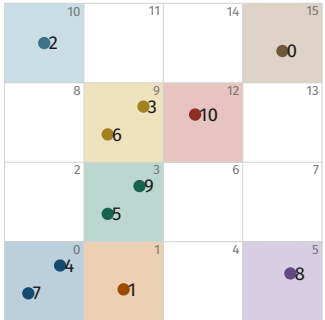
1. Ячейка точки — как в регулярной сетке: $\mathbf{c} = \lfloor (\mathbf{p} - \mathbf{b}_{\min})/h \rfloor$.
2. Биты c_x и c_y **чередуются**.





В 3D — по три бита: $z = \ldots\ z_1 y_1 x_1\ z_0 y_0 x_0$. При 10 битах на ось код занимает 30 бит.

У всех точек ячейки один Z-индекс

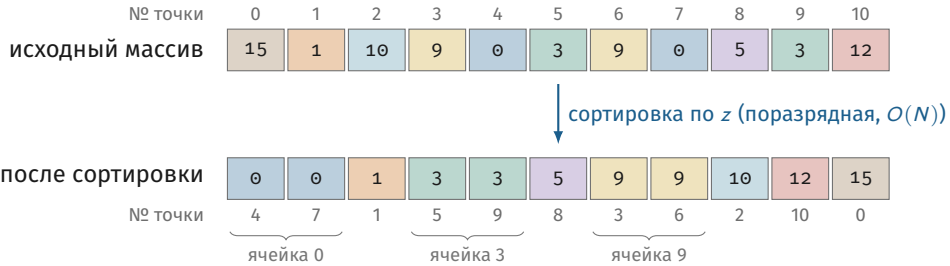


точка	$\mathbf{p}$	ячейка	z
6	$(1{,}30;\ 2{,}35)$	$(1, 2)$	9
3	$(1{,}75;\ 2{,}70)$	$(1, 2)$	9
5	$(1{,}30;\ 1{,}35)$	$(1, 1)$	3
9	$(1{,}70;\ 1{,}70)$	$(1, 1)$	3

Дробная часть координат отбрасывается при вычислении ячейки, поэтому z зависит только от ячейки.

Z-индекс — это номер ячейки, просто в другом порядке обхода.

Сортировка по Z-индексу

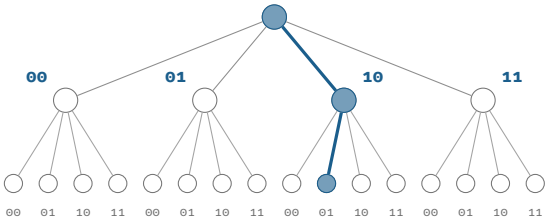


- Точки одной ячейки стоят подряд: ячейка — это отрезок массива, его граница — там, где меняется z.
- Не нужен ни массив на все ячейки, ни хэш-таблица: только отсортированный массив индексов, память N чисел (0, если переставить сами точки).
- Соседние ячейки обычно рядом и в массиве: меньше промахов кэша.

Z-индекс — это путь в дереве

1010	**10**11	**11**10	**11**11
1000	**10**01	**11**00	**11**01
0010	**00**11	**01**10	**01**11
0000	**00**01	**01**00	**01**01

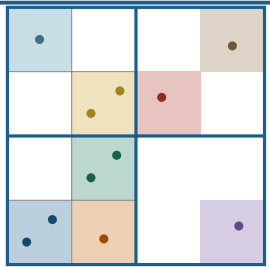
Первые 2 бита — квадрант, следующие 2 — подквадрант в нём.



ячейка **10**01: корень → 10 → 01

Префикс Z-индекса — это узел квадродерева. В 3D по 3 бита на уровень, 8 потомков — октодерево.

Строим дерево по отсортированному массиву



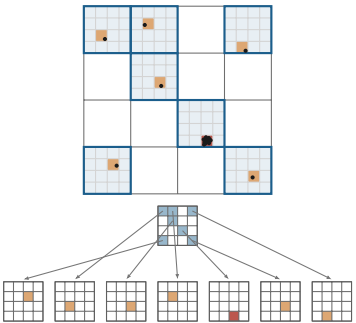
Квадранты 00 и 10 разбиты, 01 и 11 — листья.



- Узел — отрезок массива с общим префиксом z. Потомки — куски, на которых следующие 2 бита (в 3D — 3) одинаковы; их границы находит двоичный поиск.
- Делим, пока в узле больше K точек (здесь $K = 2$). Пустые квадранты не появляются, а дерево глубже там, где точек больше.

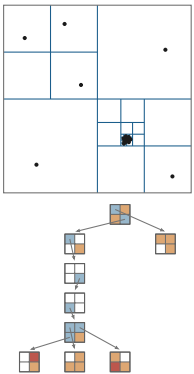
Иерархические сетки и октодеревья

Иерархическая сетка (узел 4×4, в 3D — 16^3)



Короткое широкое дерево, но все 18 точек кластера — в одной ячейке.

Квадродерево (узел 2×2, в 3D — октодерево)



Делим, пока в узле больше 2 точек: дерево высокое и узкое, глубже только в кластере.

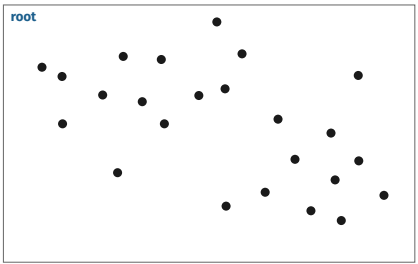
есть потомок | 1–2 точки | больше 2 точек | пусто

4

РАЗДЕЛ

kd-деревья и BSP

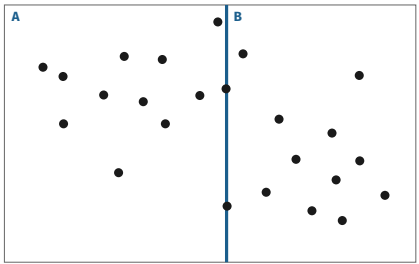
kd-дерево: построение

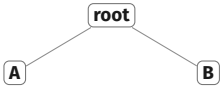


root

- Узел — бокс. Делим его **плоскостью, перпендикулярной оси**; ось чередуется: $x, y, z, x, \ldots$
- Плоскость можно поставить куда угодно, например по медиане: поровну точек слева и справа.
- Останавливаемся, когда в листе $\leq K$ точек. В k-мерном пространстве всё то же самое.

kd-дерево: построение

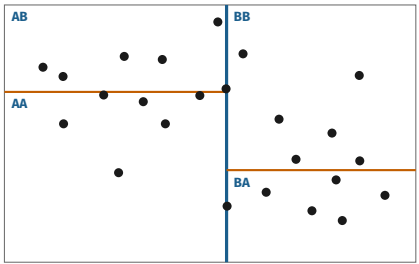


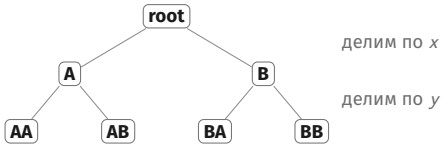


делим по x

- Узел — бокс. Делим его **плоскостью, перпендикулярной оси**; ось чередуется: $x, y, z, x, \ldots$
- Плоскость можно поставить куда угодно, например по медиане: поровну точек слева и справа.
- Останавливаемся, когда в листе $\leq K$ точек. В k-мерном пространстве всё то же самое.

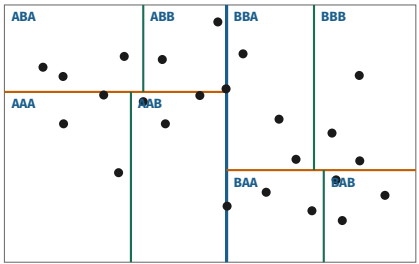
kd-дерево: построение

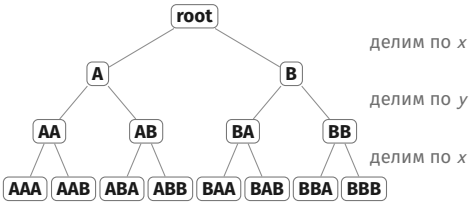




- Узел — бокс. Делим его **плоскостью, перпендикулярной оси**; ось чередуется: $x, y, z, x, \ldots$
- Плоскость можно поставить куда угодно, например по медиане: поровну точек слева и справа.
- Останавливаемся, когда в листе $\leq K$ точек. В k-мерном пространстве всё то же самое.

kd-дерево: построение



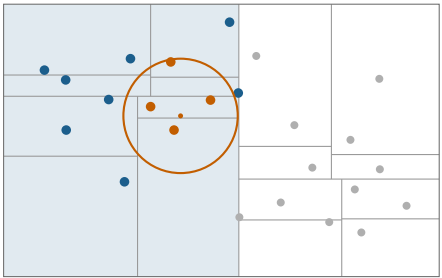


- Узел — бокс. Делим его **плоскостью, перпендикулярной оси**; ось чередуется: $x, y, z, x, \ldots$
- Плоскость можно поставить куда угодно, например по медиане: поровну точек слева и справа.
- Останавливаемся, когда в листе $\leq K$ точек. В k-мерном пространстве всё то же самое.

kd-дерево: поиск точек в шаре

```
void query(int i) {
  Node n = nodes[i];
  if (n.isLeaf) {            // перебор листа
    for (int k = n.begin; k < n.end; k++)
      check(k);
    return;
  }
  float d = q[n.axis] - n.split;
  if (d <  r) query(n.left);  // шар задевает
  if (d > -r) query(n.right); // одну или обе
}
```

- Шар целиком с одной стороны плоскости — спускаемся в одного потомка.
- Плоскость режет шар — в обоих.



Делим до $K = 2$ точек в листе. Запрос посетил 8 листьев из 16, проверил 12 точек, в шаре 4.

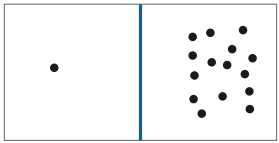
Спуск — $O(\log N)$ сравнений, по одному на уровень. Плотность точек больше не важна: дерево глубже там, где точек больше.



Как выбирать плоскость разбиения?

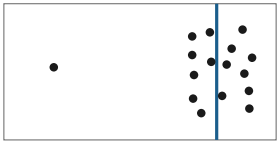
kd-дерево: где резать?

По центру



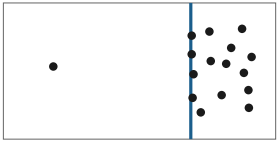
режем пространство пополам

По медиане



поровну точек: дерево сбалансировано

По стоимости (SAH)



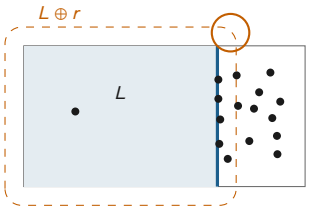
отрезаем пустое пространство

- Одинокая точка и плотный кластер. Под рисунком — число точек слева и справа.
- **Центр**: в правой половине почти все точки, а в левой много пустоты.
- **Медиана**: сбалансированное дерево, но плоскость режет кластер, и запросы в кластере посещают обе половины.
- **SAH**: пустое пространство отсекается сразу, запрос в пустую область заканчивается на первом же уровне.

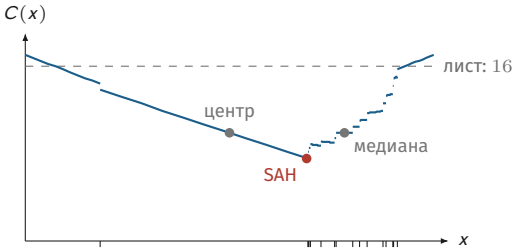
Surface Area Heuristic (SAH)

Выбираем x, минимизируя ожидаемую стоимость запроса в узле X:

$$C(x) = C_{\text{trav}} + C_{\text{test}}\big(P_L\, N_L(x) + P_R\, N_R(x)\big), \qquad P_L = \frac{V(L \oplus r)}{V(X \oplus r)}.$$



Шар радиуса r задевает L, если его центр попал в $L \oplus r$ — бокс, раздутый на r.



точки рисунка слева: засечки по оси x

- Для лучей $P_L = S(L)/S(X)$ — отношение площадей поверхности, отсюда название.
- Минимум ищем среди координат точек; узел не делим, если $\min C(x) > C_{\text{test}} N$.

kd-дерево: компактный узел

```
struct KdNode {                    // 8 байт
  uint32_t flagAxisOffset;  // биты 0-1: ось
                            // биты 2-30: левый потомок
                            // бит 31: лист
  float    split;           // координата плоскости
};                          // правый = левый + 1
```

Память

Листьев $\approx N/K$, всего узлов $< 2N/K$: при $K = 8$ это 2 байта на точку.

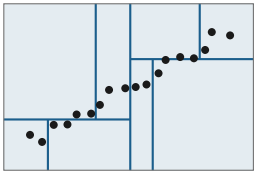
У листа те же 8 байт: начало и число его точек. Точки переставлены в порядке листьев, индексы не нужны.

- \+ Адаптивность: «чайник на стадионе» не страшен.
- \+ Компактно: 8 байт на узел.
- \+ Простой поиск, почти не зависит от размерности.

- – На GPU поиску нужен стек.
- – Хорошее (SAH) построение нетривиально, $O(N \log N)$.
- – Данные изменились — дерево надо перестраивать.

kd-дерево и BSP-дерево

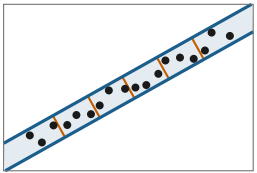
kd-дерево: плоскости вдоль осей



7 разбиений, а листья — «лесенка» с пустыми углами.

+ Узел: ось и число, тест — одно сравнение.
+ SAH-дерево строится за $O(N \log N)$.
– Наклонная геометрия разбивается плохо.

BSP-дерево: плоскости любой ориентации



Две плоскости отсекают всю пустоту, листья плотно облегают точки.

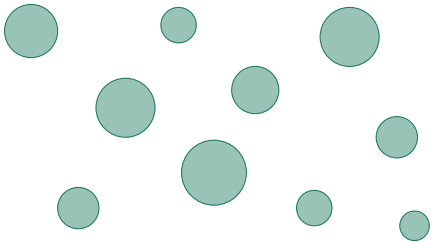
+ Теоретически эффективнее: меньше узлов.
– Узел — плоскость $(\mathbf{n}, d)$, 16 байт; тест — скалярное произведение.
– Ячейки — многогранники; SAH-построение очень долгое.

5

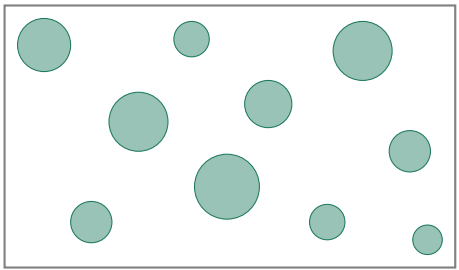
РАЗДЕЛ

BVH

BVH: иерархия ограничивающих объёмов



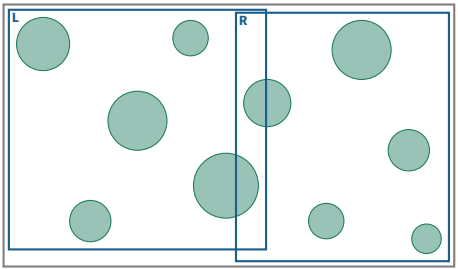
BVH: иерархия ограничивающих объёмов

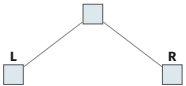




- Делим объекты, а не пространство: корень — бокс всех объектов, потомки — их половины.

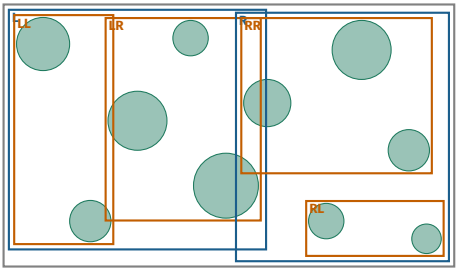
BVH: иерархия ограничивающих объёмов

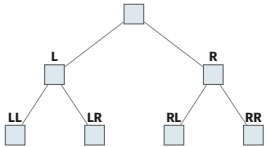




- Делим объекты, а не пространство: корень — бокс всех объектов, потомки — их половины.

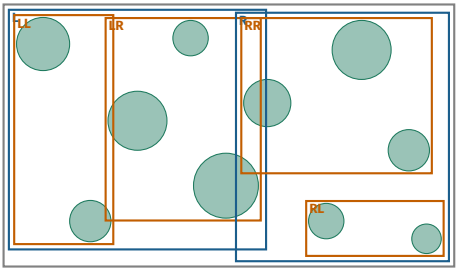
BVH: иерархия ограничивающих объёмов

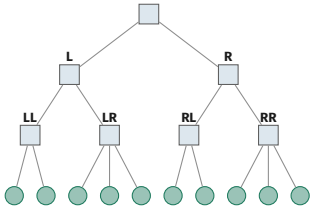




- Делим объекты, а не пространство: корень — бокс всех объектов, потомки — их половины.
- Каждый объект — ровно в одном листе, дерево строится вниз, пока в узле больше 3 объектов.

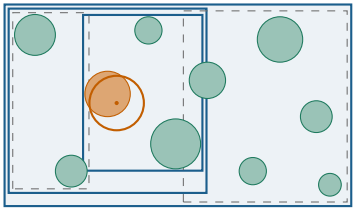
BVH: иерархия ограничивающих объёмов

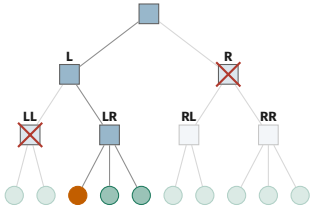




- Делим объекты, а не пространство: корень — бокс всех объектов, потомки — их половины.
- Каждый объект — ровно в одном листе, дерево строится вниз, пока в узле больше 3 объектов.
- Узел хранит бокс своих объектов. Боксы соседей могут пересекаться.

BVH: поиск объектов, задевающих шар





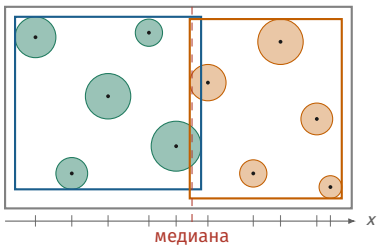

```
void query(int i) {
  Node n = nodes[i];
  if (dist2(q, n.box) > r*r) return; // мимо бокса
  if (n.isLeaf) { checkObjects(n); return; }
  query(n.left);                   // всегда оба:
  query(n.right);                  // боксы пересекаются
}
```

Бокс мимо шара — отбрасываем всё поддерево (×). Проверено объектов: 3, задевают шар: 1 из 10.

В отличие от kd-дерева спускаемся в обоих потомков, если шар задевает оба бокса.

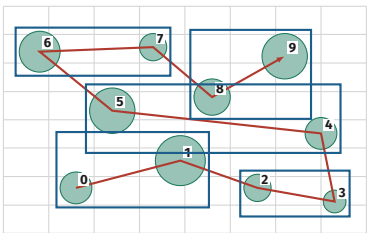
Построение BVH

Сверху вниз



- Сортируем центры по длинной оси бокса.
- Делим по медиане или по SAH, рекурсивно.
- $O(N \log N)$, лучшее качество дерева.

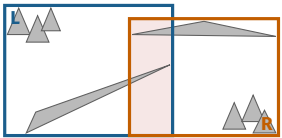
Снизу вверх: LBVH



- Сортируем центры по Z-индексу (раздел 3).
- Соседи в массиве близки и в пространстве: дерево над массивом.
- $O(N)$ с поразрядной сортировкой, строится на GPU за миллисекунды.

Object split и spatial split

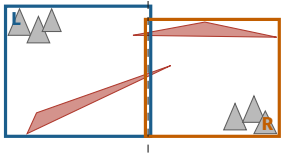
Object split: делим примитивы



Примитив целиком в одном потомке; боксы пересекаются.

- \+ Одна ссылка на примитив: память известна заранее.
- \+ Простое построение, быстрый refit.
- – Пересечение в листе обход не останавливает.
- – Длинные треугольники раздувают боксы.

Spatial split: делим пространство



Плоскость режет примитивы: они попадают в оба потомка.

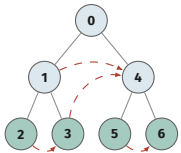
- \+ Потомки не пересекаются: стоп на первом попадании.
- \+ Плотные боксы на длинной геометрии.
- – Дубликаты: память заранее неизвестна.
- – Сложнее строить; refit не работает.

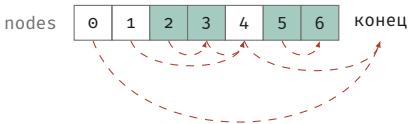
SBVH [Stich 2009] смешивает оба: выбор по SAH в каждом узле.



Эквивалентны ли object split и spatial split, если внутри дерева — точки?

BVH: обход без стека





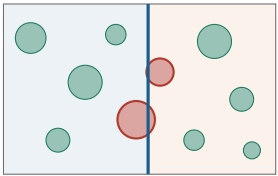
Узлы лежат в порядке обхода вглубь. Красный пунктир — `skip`: куда идти, если поддерево не нужно; у листа это **верёвка** (rope) к соседу.

```
int i = 0;
while (i < numNodes) {
  Node n = nodes[i];
  bool hit = dist2(q, n.box) <= r*r;
  if (hit && n.isLeaf) checkObjects(n);
  if (hit && !n.isLeaf) i = i + 1;  // вглубь
  else                  i = n.skip; // мимо
}
```

- Порядок обхода фиксирован: всегда слева направо.
- Состояние — одно число `i`: на GPU не нужен стек.
- У листа `skip = i+1` — «верёвка» к следующему узлу обхода.

BVH и kd-дерево

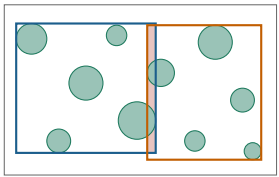
kd-дерево: делим пространство



Объекты на плоскости попадают в оба потомка: дубликаты.

+ Ячейки не пересекаются: спуск в одного потомка.
+ Узел 8 байт.
– Дубликаты; объект сдвинулся — перестроить.

BVH: делим объекты

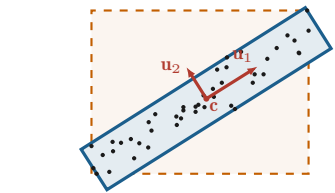


Каждый объект в одном листе, но боксы пересекаются.

+ Без дубликатов, память $O(N)$ предсказуема.
+ Объекты сдвинулись — пересчитать боксы (refit).
– Узел крупнее (бокс 24 байта); запрос может пойти в обоих потомков.

BVH — стандарт трассировки лучей (RT-ядра GPU). Гибрид BIH: две плоскости в узле.

Ориентированный бокс (OBB)



Бокс повёрнут вдоль объекта: центр $\mathbf{c}$, оси $\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3$, полуразмеры e_i.

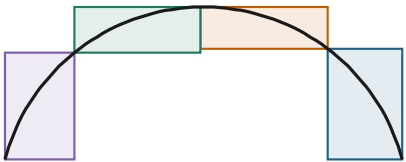
1. Оси — собственные векторы **ковариации** точек (PCA): $\mathbf{u}_1$ — направление наибольшего разброса.
2. Проецируем точки на оси, берём min и max — получаем e_i.

Запрос шаром: перевести $\mathbf{q}$ в систему бокса, $\mathbf{q}' = R^{\mathsf{T}}(\mathbf{q} - \mathbf{c})$, и дальше — как для AABB.

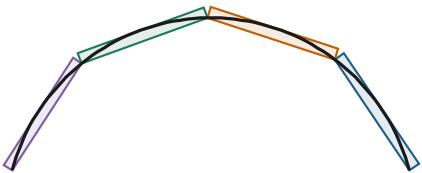
Цена: 15 чисел вместо 6 и поворот при каждой проверке.

OBB-дерево

4 AABB: площадь 6,47



4 OBB: площадь 1,4



- BVH из OBB — **OBB-дерево**. На кривых поверхностях зазор OBB убывает квадратично с размером узла, у AABB — линейно [Gottschalk 1996].
- Столкновение двух OBB — проверка 15 разделяющих осей, меньше 200 операций. Отсюда применение: обнаружение столкновений.
- В трассировке лучей BVH из AABB можно перестроить в OBB-дерево на GPU [Vitsas 2023]: меньше лишних проверок, но узел крупнее.

k-DOP: многогранник с фиксированными нормалями

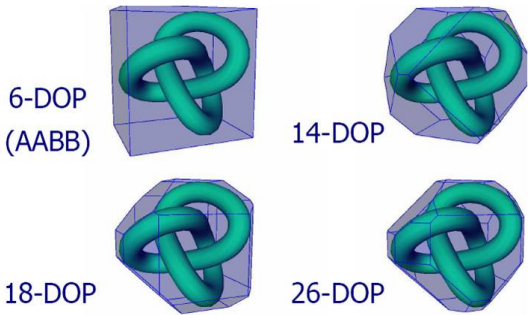


Рисунок из статьи В. Фролова «BVH», ray-tracing.ru.

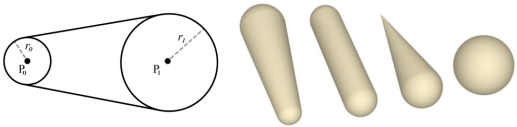
- $k/2$ заранее заданных направлений, для каждого — min и max.
- 6-DOP = AABB; 14: оси + 4 диагонали куба; 18: оси + 6 диагоналей граней; 26: все 13.
- Луч — тот же slab-тест, что у AABB, по $k/2$ направлениям.

```
struct DOP14 { float lo[7], hi[7]; };
bool overlap(DOP14 a, DOP14 b) {
  for (int i = 0; i < 7; i++)  // 7 интервалов
    if (a.hi[i] < b.lo[i] || b.hi[i] < a.lo[i])
      return false;
  return true;
}
```

SSV: сфера, «протянутая» по ядру

Vulkan:

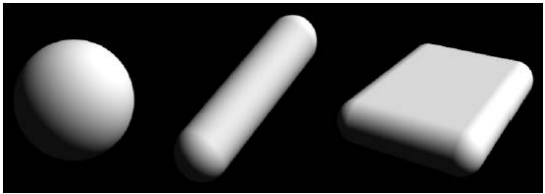
`VK_NV_ray_tracing_linear_swept_spheres`





- Сфера и **LSS** — встроенная геометрия BLAS.
- LSS: конус или капсула, радиусы r_0, r_1; цепочка LSS — волос.
- RTX 50: волосы $\approx 2\times$ быстрее, памяти $\approx 5\times$ меньше, чем у треугольников.

Рисунки: NVIDIA Technical Blog, 2025.



PSS, LSS, RSS. Рисунок из статьи В. Фролова «BVH», ray-tracing.ru.

- Все точки на расстоянии $\leq r$ от **ядра**: точки, отрезка или прямоугольника.
- Расстояние до SSV = до ядра $- r$: столкновения — через ядра [Larsen 2000].
- Бокс, раздутый на r, со слайда про SAH — тоже SSV.

Какой ограничивающий объём выбрать?

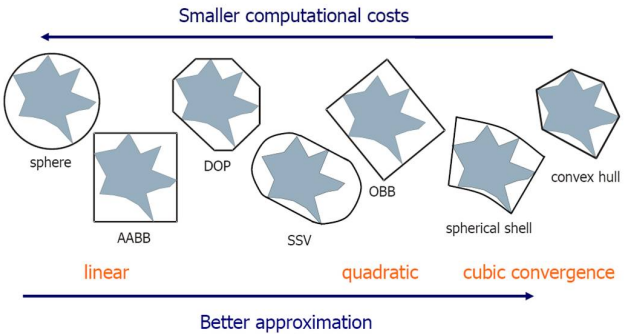


Рисунок из статьи В. Фролова «BVH», ray-tracing.ru.

- Вправо — точнее: зазор до поверхности убывает линейно, квадратично, кубически.
- Влево — дешевле тест и меньше памяти на узел.
- Точность — в метрике Хаусдорфа: $d_H(X,Y) = \max\{h(X,Y), h(Y,X)\}$, $h(X,Y) = \sup_{x\in X} \inf_{y\in Y} |x-y|$.
- Сходимость зависит от геометрии. Дома и комнаты хорошо приближаются боксами: на практике почти всегда AABB.

6

РАЗДЕЛ

Библиотеки

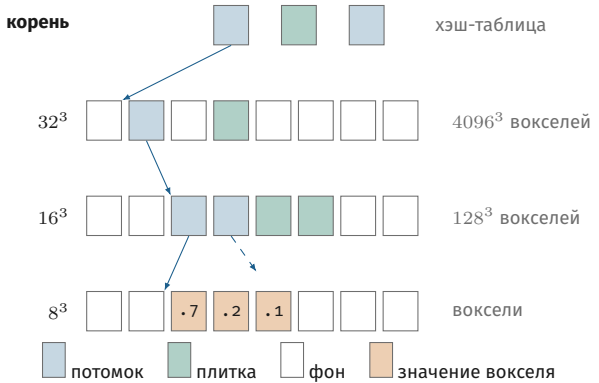
OpenVDB: короткое широкое дерево

Задача: разреженные объёмы — level set поверхности, дым, облака точек. Данные занимают узкую полосу, а область огромна и растёт.

Октодереву для 8192^3 нужно 13 уровней. VDB берёт мало уровней с большим ветвлением, как B+-дерево:

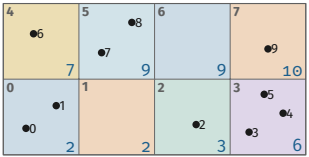
- корень — хэш-таблица, область не ограничена;
- два уровня плотных узлов 32^3 и 16^3;
- лист — плотный блок 8^3 вокселей.

Вместо потомка можно хранить **плитку** — одно значение на всю подобласть.

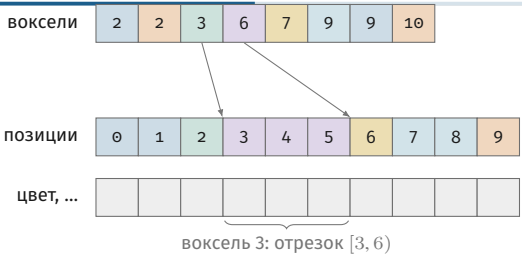


K. Museth. VDB: High-Resolution Sparse Volumes with Dynamic Topology. ACM TOG, 2013.

OpenVDB Points: точки в листьях



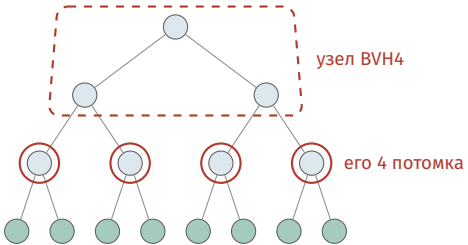
Лист в 2D: 8 вокселей. Синее число в углу — значение вокселя: конец его отрезка в массиве точек.



- `PointDataGrid`: точки разложены по листьям, в листе — по вокселям: схема «смещений» из раздела 1, но только в непустых листьях.
- Позиция — относительно центра вокселя (можно 16 бит), атрибуты — отдельные массивы.

Запрос шаром: дерево → листья, задевающие бокс шара → воксели → отрезки точек.

Embree: широкая BVH



бинарная BVH: 3 уровня ⟶ 2

```
struct BVH4Node {       // 4 бокса — SoA
  float lower_x[4], upper_x[4];
  float lower_y[4], upper_y[4];
  float lower_z[4], upper_z[4];
  NodeRef child[4];
};
// шар против 4 боксов — одной SIMD-командой
```

- Intel Embree — ядра трассировки лучей на CPU (Blender Cycles, Corona, …).
- Узел на 4 (8) потомков: глубина вдвое меньше, 4 бокса — одной SIMD-инструкцией.
- Построение — SAH; есть поиск в радиусе: `rtcPointQuery` с колбэком пользователя.
- На GPU та же идея в железе: RT-ядра обходят BVH (OptiX, Vulkan/DirectX Ray Tracing).

Библиотеки: что внутри

Библиотека	Структура	Где применяется
cuNSearch, SPH-движки	регулярная сетка + сортировка	частицы, жидкости
Open3D (GPU)	пространственный хэш	поиск в радиусе на GPU
nanoflann, FLANN, CGAL	kd-дерево	облака точек, kNN и радиус
PCL	kd-дерево (FLANN), октодерево	лидары, сжатие облаков
OpenVDB, NanoVDB	иерарх. сетка + хэш в корне	объёмы, level set, точки
Embree	широкая BVH (4/8), SAH	трассировка лучей на CPU
OptiX, Vulkan RT	BVH в железе (RT-ядра)	трассировка лучей на GPU
Bullet, Box2D	динамическое AABB-дерево (BVH)	столкновения

Точки и фиксированный радиус — сетка или хэш. Неравномерная плотность — kd- или октодерево. Объекты с размером и лучи — BVH. Разреженные объёмы — VDB.

7

РАЗДЕЛ

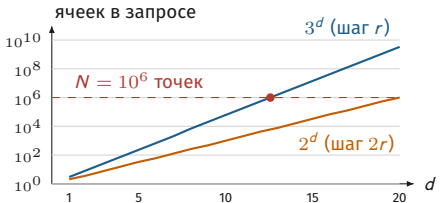
Высокие размерности



Что станет с этими структурами
в пространстве размерности 100?

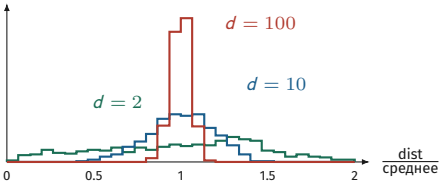
Проклятие размерности

Сетка: ячеек в запросе шаром



Уже при $d \approx 13$ запрос проверяет больше ячеек, чем точек в облаке.

Расстояния до 2000 случайных точек

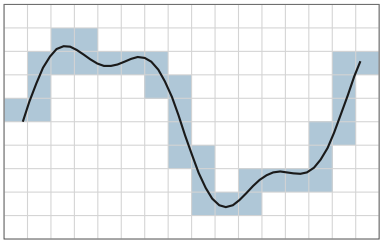


Разброс $(d_{\max} - d_{\min})/d_{\min}$: 72 ($d = 2$), 3,1 ($d = 10$), 0,45 ($d = 100$).

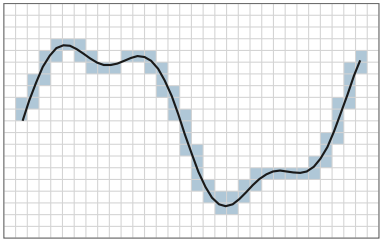
- Ближайшая точка почти так же далеко, как самая дальняя [Beyer 1999]: отсечение «бокс заведомо дальше» не срабатывает.
- С $d \approx 10$ деревья и сетки проигрывают перебору [Weber 1998]; kd-дереву нужно $N \gg 2^d$.
- Но всё это — для точек, равномерно заполняющих пространство. Реальные данные устроены иначе.

Внутренняя размерность

шаг h: занято 32 из 160 ячеек

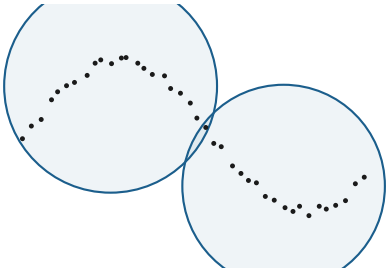


шаг $h/2$: занято 64 из 640 ячеек



- Ячеек стало в 4 раза больше, а занятых — примерно в 2,0 раза: данные ведут себя как одномерные.
- Число занятых ячеек растёт как $(1/h)^{d_{\text{вн}}}$, где $d_{\text{вн}}$ — **внутренняя размерность**. У облака лидара (поверхность) она около 2, у текстовых эмбеддингов GloVe — около 20 [Wang 2021], а не сотни.
- Поиск остаётся трудным, но не настолько, как для равномерного куба.

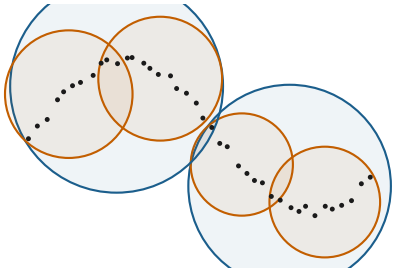
Ball tree: BVH из шаров



Точки на кривой: шары следуют за данными, а не за осями координат.

- Узел — шар (центр и радиус) вокруг своих точек, шары могут пересекаться: это BVH со сферами [Omohundro 1989].
- Проверка узла — одно расстояние до центра. Нужна только метрика, координаты не обязательны: VP-tree, метрические деревья [Uhlmann 1991, Yianilos 1993].
- **Cover tree**: запрос за $O(c^{12} \log n)$, где c — мера внутренней размерности [Beygelzimer 2006].
- Лучшие точные методы, пока внутренняя размерность мала. В scikit-learn kd-дерево — до $D \approx 15$–20, дальше ball tree или перебор.

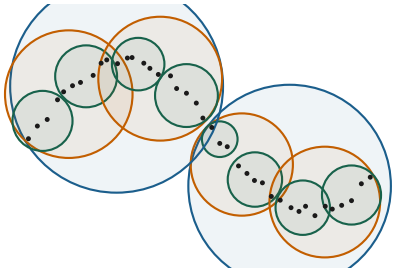
Ball tree: BVH из шаров



Точки на кривой: шары следуют за данными, а не за осями координат.

- Узел — шар (центр и радиус) вокруг своих точек, шары могут пересекаться: это BVH со сферами [Omohundro 1989].
- Проверка узла — одно расстояние до центра. Нужна только метрика, координаты не обязательны: VP-tree, метрические деревья [Uhlmann 1991, Yianilos 1993].
- **Cover tree**: запрос за $O(c^{12} \log n)$, где c — мера внутренней размерности [Beygelzimer 2006].
- Лучшие точные методы, пока внутренняя размерность мала. В scikit-learn kd-дерево — до $D \approx 15$–20, дальше ball tree или перебор.

Ball tree: BVH из шаров

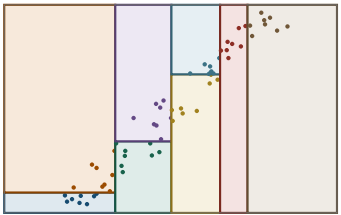


Точки на кривой: шары следуют за данными, а не за осями координат.

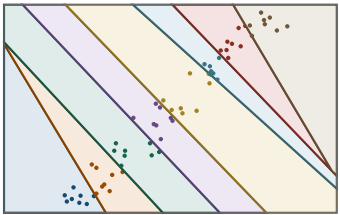
- Узел — шар (центр и радиус) вокруг своих точек, шары могут пересекаться: это BVH со сферами [Omohundro 1989].
- Проверка узла — одно расстояние до центра. Нужна только метрика, координаты не обязательны: VP-tree, метрические деревья [Uhlmann 1991, Yianilos 1993].
- **Cover tree**: запрос за $O(c^{12} \log n)$, где c — мера внутренней размерности [Beygelzimer 2006].
- Лучшие точные методы, пока внутренняя размерность мала. В scikit-learn kd-дерево — до $D \approx 15$–20, дальше ball tree или перебор.

PCA-дерево: режем поперёк главной оси

kd-дерево: разрезы вдоль осей



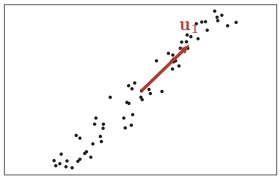
PCA-дерево: разрез поперёк $\mathbf{u}_1$



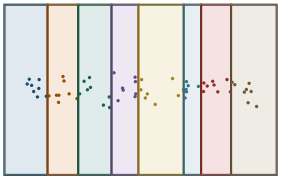
Три уровня, разрез по медиане. Средний диаметр ячейки: kd — 1,02, PCA — 0,89. В 2D выигрыш скромный, в высокой размерности он растёт.

- OBB в размерности d хранит матрицу $d \times d$: при $d = 768$ это 590 тыс. чисел на узел. Поэтому узел хранит одно направление: главную ось своих точек — principal axis tree [McNames 2001], или случайное направление — RP-tree [Dasgupta, Freund 2008].
- PCA- и 2-means-деревья лучше всех подстраиваются под внутреннюю размерность, kd-дерево — хуже [Verma 2009].

Повернуть один раз: PCA перед kd-деревом





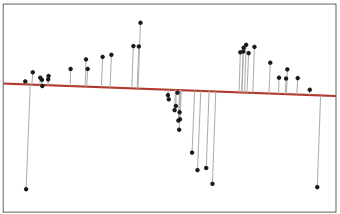


kd-дерево после поворота: средний диаметр ячейки 0,90 (было 1,02).

- Вместо поворота в каждом узле — один поворот всего набора: это OBB для всех данных сразу.
- Сопоставление SIFT: PCA-поворот и лес случайно повёрнутых kd-деревьев [Silpa-Anan, Hartley 2008]; так устроен FLANN.
- OPQ: поворот перед product quantization, чтобы куски вектора несли поровну информации [Ge 2013].

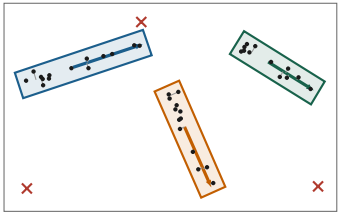
LDR: свой PCA для каждого кластера

GDR: одна главная ось на всех



Серые отрезки — потеря при проекции; в среднем 0,78.

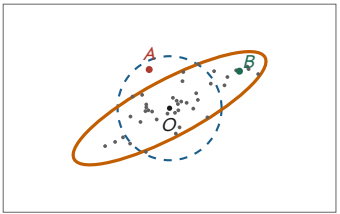
LDR: своя ось у каждого кластера



Полоса — допуск `MaxReconDist`; потеря в среднем 0,07. × — выбросы.

- Пространственная кластеризация → PCA в каждом кластере → точки перераспределяются: точка идёт в кластер, где расстояние до подпространства $\leq$ `MaxReconDist`.
- У каждого кластера своя размерность d_i: минимальная, при которой порог нарушает лишь малая доля точек.
- Не попавшие ни в один кластер — выбросы, их просто перебирают [Chakrabarti, Mehrotra 2000].

MMDR: эллипсы вместо шаров



Пунктир — шар по Евклиду, сплошная — эллипс по Махаланобису. По Евклиду ближе A, по Махаланобису — B.

$$\mathrm{MahaDist}(P, O) = (P - O)^{\mathsf{T}} C^{-1} (P - O)$$



Ключ $y = i \cdot c + \mathrm{dist}(P, O_i)$: все кластеры в одном B$^+$-дереве (расширенный iDistance).

- Кластеры ищет эллиптический k-means в проекциях малой размерности; если ошибка проекции велика, размерность проекции удваивается. Собственные векторы C — те же оси PCA.
- У каждого эллипсоида своя система координат и своя размерность; вставки — слиянием пересекающихся эллипсоидов.
- Синтетика, 20 оставленных измерений: совпадение ответа с точным у LDR не выше 60 %, у глобального PCA — 25 %, у MMDR выше [Shen, Zhou, Zhou 2007].

8

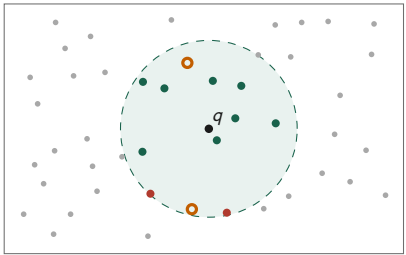
РАЗДЕЛ

Векторный поиск



Почему тогда работают векторные базы данных?

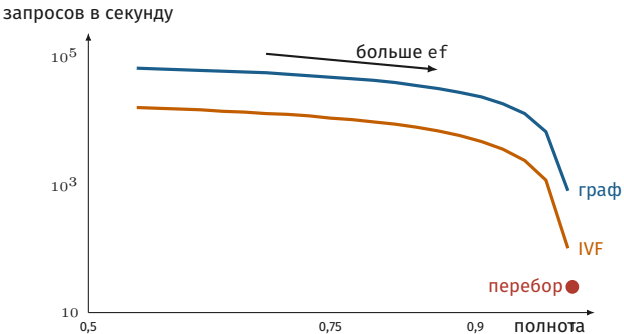
Другая постановка: k ближайших, приближённо



- Объекты — **эмбеддинги**: картинка или текст → вектор из сотен–тысяч чисел.
- Вместо шара — k ближайших (top-k): радиус заранее неизвестен.
- Метрика — часто косинус. Для векторов длины 1 $\|\mathbf{a}-\mathbf{b}\|^2 = 2 - 2\,\mathbf{a}\cdot\mathbf{b}$: это тот же евклидов поиск.
- Ответ приближённый. Качество — **полнота**: $\text{recall@}k = \dfrac{|\text{найдено} \cap \text{истинные}|}{k}$. На рисунке $8/10$.

Нижние оценки из прошлого раздела — про точный ответ в худшем случае. Приближённый поиск на реальных данных им не подчиняется.

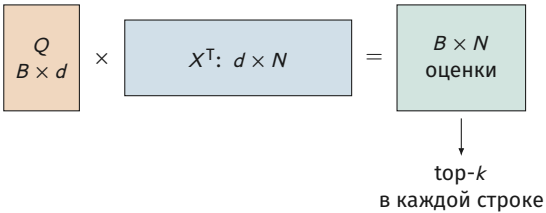
Полнота против скорости



Настоящие кривые — на ANN-Benchmarks [Aumüller 2019].

- Индекс — это не точка, а кривая: параметр запроса (`nprobe`, `ef`) двигает по ней.
- Хотим больше полноты — платим скоростью.
- Перебор — одна точка: полнота 1, скорость низкая.
- Методы сравнивают так: кто быстрее при полноте 0,9 или 0,95.

Перебор — это умножение матриц



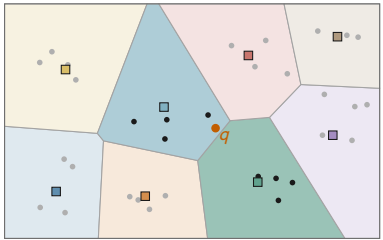
Пример: $N = 10^6$ векторов, $d = 768$, `float32`.

- Данные: 3,1 ГБ. Один запрос: 1,5 GFLOP.
- Пачка из 1000 запросов на GPU (~50 TFLOP/s): ≈31 мс, то есть ~33 тыс. запросов в секунду — точно.
- Одиночный запрос упирается в чтение 3,1 ГБ: ≈1,5 мс при 2 ТБ/с.

Оценка по порядку величины. Точный перебор на GPU — в FAISS [Johnson 2019].

Перебор — честная точка отсчёта. Индекс нужен, когда векторов миллиарды, нет GPU или важна задержка одного запроса.

IVF: сетка, у которой ячейки строит k-means

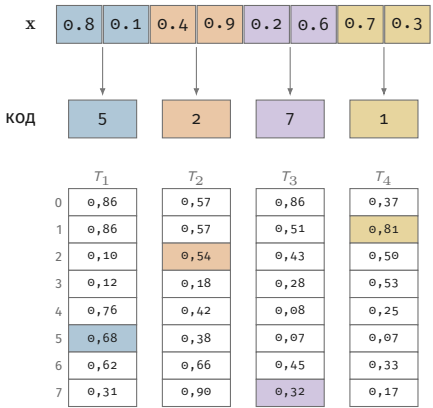


Яркие — ячейки, которые просматривает запрос (`nprobe = 2`).

- **Построение**: k-means даёт k центров (квадратики). Ячейка — все точки, ближайшие к центру (**ячейка Вороного**).
- Точки хранятся по ячейкам подряд, как в способе со смещениями из раздела 1.
- **Запрос**: найти `nprobe` ближайших центров и перебрать их списки.
- Сосед может лежать в соседней ячейке: `nprobe` — ручка «полнота — скорость».

IVF — это нерегулярная сетка, подстроенная под данные: ячейки там, где точки.

PQ: вектор — несколько байт



8 чисел: 32 байта

4 номера: 4 байта

$$d^2(\mathbf{q}, \mathbf{x}) \approx 0{,}68 + 0{,}54 + 0{,}32 + 0{,}81 = 2{,}35$$

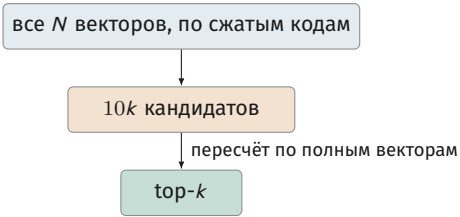
1. Режем вектор на куски по 2 числа (цвета).
2. Для каждого куска свой k-means на 256 центров; храним номер ближайшего центра. В примере для краткости центров 8.
3. Запрос: для каждого куска заранее считаем расстояния до всех его центров — таблицы T_m.
4. Расстояние до точки — 4 чтения из таблиц и сумма, без умножений.

Product quantization [Jégou 2011]. При $d = 768$ и 96 кусках вектор занимает 96 байт вместо 3072.

Сжатие векторов и пересчёт лучших

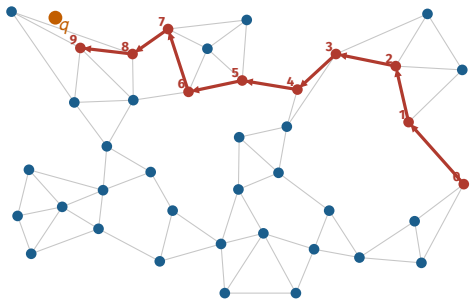
Вектор $d = 768$:

формат	байт	сжатие	расстояние
`float32`	3072	1×	точное
`float16`	1536	2×	почти точное
`int8` (скалярное)	768	4×	целочисленное
PQ, 96 кусков	96	32×	по таблицам
1 бит на число	96	32×	Хэмминг (`popcount`)



- Грубые коды быстро отсеивают почти всё; немногих кандидатов пересчитываем точно.
- Бинарное квантование стало обычной практикой: в Elasticsearch оно по умолчанию для $d \geq 384$; RaBitQ даёт 1 бит на число с гарантией ошибки [Gao 2024].

Граф близости: жадный поиск



Каждая точка соединена с 3 ближайшими. Путь 0 → 9 — 9 шагов от дальнего входа.

1. Начинаем с точки входа.
2. Смотрим соседей текущей точки и переходим к тому, кто ближе к q.
3. Ни один сосед не ближе — стоп: это ответ.

- Нет ни ячеек, ни боксов: только «кто кому сосед».
- На деле держат не одного, а `ef` лучших кандидатов — так реже застревают.
- Минус: от дальнего входа идти долго.

Что выбрать

Задача	Структура	Аналог в лекции
$d \leq 3$–4, запрос шаром	сетка, хэш, kd, BVH, VDB	вся лекция
$d \approx 5$–20, $N \gg 2^d$	kd-дерево, ball tree	kd, BVH
d велико, внутренняя мала, нужен точный ответ	ball tree, cover tree	BVH со сферами
точный ответ, есть GPU	перебор пачками	«наивное решение»
k ближайших, $d \sim 10^2$–10^3, в памяти	граф HNSW	—
мало памяти	IVF + PQ или бинарное квантование	сетка со смещениями + сжатие
10^9 векторов	DiskANN; IVF-PQ на GPU	—
фильтры по атрибутам	граф с доп. рёбрами	—